

GUI-Based Multi-Client Chat Application Using TCP

NAME: Kaushal Kumar Roll : CSB24069

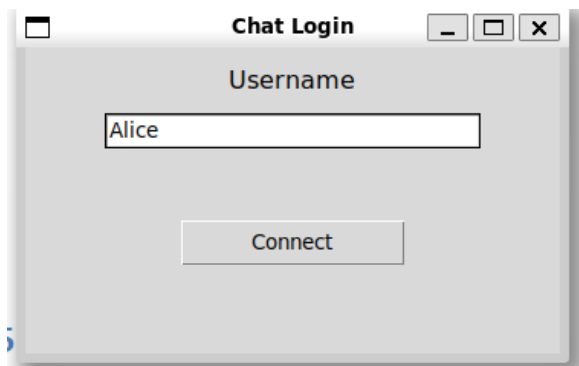
1. Objective

Convert the terminal-based TCP chat application developed in the previous assignment into a graphical desktop application while reusing the existing server implementation. The objective is to introduce GUI programming, event-driven programming, multithreading, and user-friendly network application development.

2. GUI Design

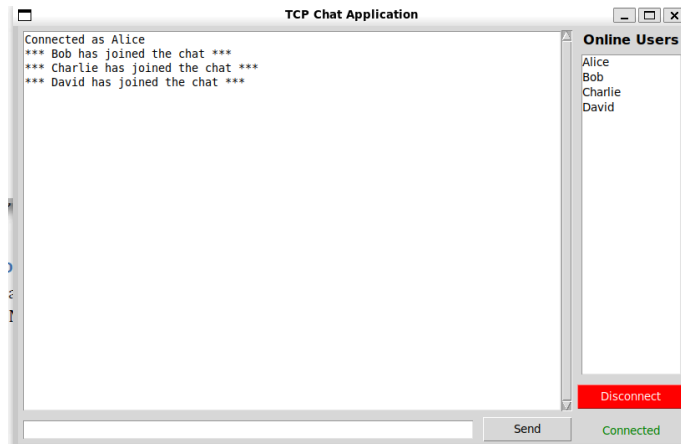
The graphical user interface (GUI) of the chat application was developed using the **Python Tkinter** library to provide a simple, interactive, and user-friendly environment for communication. The interface was designed according to the assignment requirements, focusing on functionality rather than appearance. The application consists of two main windows: the **Login Window** and the **Main Chat Window**.

2.1 Login Window



After successful validation and connection, the Login Window is hidden, and the Main Chat Window is displayed.

2.2 Main Chat Window



2.3 GUI Features

The implemented GUI provides the following features:

- User login with username validation.
- Real-time display of chat messages.
- Automatic scrolling to the latest message.
- Broadcast and private message support.
- Dynamic online user list.
- Connection status indicator.
- Safe client disconnection.
- Responsive interface using a background thread for receiving messages.⁸

The GUI provides a clean and intuitive interface while keeping the networking logic independent from the presentation layer. This modular design improves readability, maintainability, and future extensibility of the application.

3. System Architecture

The GUI-based Multi-Client Chat Application follows a **client-server architecture** using the **TCP protocol** for reliable communication. A single server manages multiple client connections, while each client provides a graphical interface developed using **Python Tkinter**. The server acts as the central communication hub, receiving messages from connected clients and forwarding them as broadcast or private messages based on the message type.

The server maintains a list of connected clients and performs the following tasks:

- Accepts new client connections.
- Authenticates usernames.
- Maintains the list of online users.
- Processes broadcast and private messages.
- Send join and leave notifications.
- Logs chat history.
- Removes disconnected clients from the active user list.

4. Components Reused from Assignment 5

This assignment was developed by reusing the core networking implementation from Assignment 5, while replacing only the terminal-based client interface with a graphical user interface (GUI) built using Python Tkinter. Reusing the existing components reduced development time and ensured that the application retained the same communication functionality.

The following components were reused:

- TCP Socket Communication:
- Client Connection Management:
- Broadcast Messaging:
- Private Messaging:
- Online User Management:
- Join and Leave Notifications:
- Chat History and Logging

5. GUI Design Decisions

The graphical user interface was designed with the primary objective of providing a simple, user-friendly, and responsive chat application while reusing the networking logic from Assignment 5. The application consists of two main windows: a Login Window and a Chat Window. The Login Window allows users to enter their username before connecting to the server. Input validation is performed to ensure that empty usernames are not accepted, improving usability and preventing invalid connections.

After a successful connection, the Chat Window is displayed. The window is divided into two sections using Frames. The left section contains a scrollable chat area that displays all incoming and outgoing messages along with a message input box and a Send button. A ScrolledText widget was chosen because it automatically supports vertical scrolling, making it suitable for displaying long conversations.

The right section contains an Online Users list implemented using a Listbox, allowing users to view all currently connected clients. A Disconnect button enables users to safely leave the chat, while a status label displays the current connection state (Connected or Disconnected), providing immediate feedback to the user.

Overall, the GUI was intentionally kept simple and functional, following the assignment guidelines. The chosen layout and widgets provide an intuitive interface while maintaining a clear separation between the graphical interface and the networking logic, resulting in a modular and maintainable application.

6. Testing Results

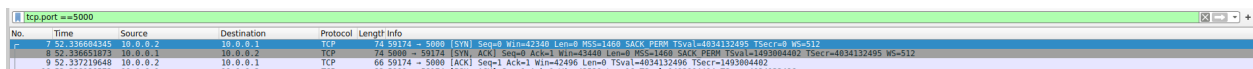
The application was tested with multiple clients communicating simultaneously through the server. All required features, including user login, broadcast messaging, private messaging, online user management, join/leave notifications, and client disconnection, functioned correctly. The background receiver thread ensured that incoming messages were processed without affecting the responsiveness of the graphical interface. The successful execution of all test cases confirms that the GUI-based chat application meets the assignment requirements and operates reliably in a multi-client TCP networking environment.

7. Wireshark Verification

Network traffic generated by the TCP chat application was captured using Wireshark with the display filter `tcp.port == 5000` to monitor all communication between the server and clients.

Client Connection

When a client connected to the server, Wireshark captured the TCP three-way handshake (SYN, SYN-ACK, ACK), followed by the exchange of the username during the login process. This verified that a reliable TCP connection was successfully established before any chat messages were transmitted.

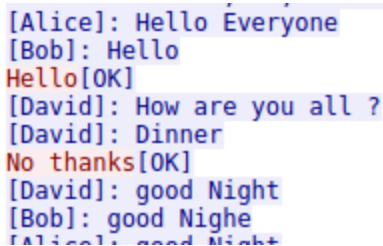


A screenshot of the Wireshark network traffic capture interface. The display filter at the top is 'tcp.port == 5000'. The packet list shows three packets: a SYN packet (No. 7), a SYN-ACK packet (No. 8), and an ACK packet (No. 9). The packet details pane shows the TCP segment information for the selected packet (No. 7), including source and destination IP addresses (10.0.0.2 and 10.0.0.1), source and destination ports (5000 and 5000), and sequence and acknowledgment numbers.

No.	Time	Source	Destination	Protocol	Length	Info
7	52.33589445	10.0.0.2	10.0.0.1	TCP	74	59174 → 5000 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM TSval=4834132495 TSecr=0 WS=512
8	52.335951873	10.0.0.1	10.0.0.2	TCP	74	5000 → 59174 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 SACK_PERM TSval=1493884402 TSecr=4834132495 WS=512
9	52.337219648	10.0.0.2	10.0.0.1	TCP	66	59174 → 5000 [ACK] Seq=1 Ack=1 Win=65535 Len=0 MSS=1460 SACK_PERM TSval=4834132496 TSecr=1493884402 WS=512

Broadcast Message

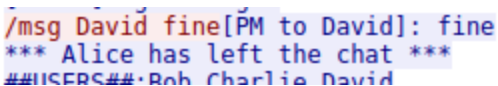
A broadcast message sent by one client was captured in Wireshark and was observed being forwarded by the server to all connected clients.



A screenshot of a chat application window showing a list of messages. The messages are: [Alice]: Hello Everyone, [Bob]: Hello, Hello[OK], [David]: How are you all ?, [David]: Dinner, No thanks[OK], [David]: good Night, [Bob]: good Nighe, and [Alice]: good Night.

Private Message

A private message sent using the `/msg <username> <message>` command was captured in the network traffic. The packet analysis showed that the server delivered the message only to the intended recipient while preventing other connected clients from receiving it.



A screenshot of a chat application window showing a list of messages. The messages are: /msg David fine[PM to David]: fine, *** Alice has left the chat ***, and ##UCERS## Bob Charlie David.

Client Disconnection

When a client clicked the Disconnect button or exited the application, Wireshark captured the TCP connection termination process using FIN and ACK packets.

335	3284.7418800	10.0.0.1	10.0.0.4	TCP	66 5000 → 5000 [FIN, ACK] Seq=395 Ack=42 Win=43520 Len=0 TSval=392768936 TSecr=392768936
336	3284.7453916	10.0.0.4	10.0.0.1	TCP	66 55038 → 5000 [FIN, ACK] Seq=42 Ack=396 Win=42496 Len=0 TSval=2286437783 TSecr=392768936

8. Reflection Answers

1. Why should networking logic and GUI code be separated?

Networking logic and GUI code should be separated to improve code organization, readability, and maintainability. The networking module is responsible for socket communication, while the GUI handles user interaction and display. Separating these components makes it easier to debug, update, and reuse the networking code without modifying the graphical interface.

2. Why is a background thread required?

A background thread is required because the `recv()` socket function is blocking. If message receiving is performed in the main GUI thread, the application becomes unresponsive while waiting for incoming messages. Running the receive operation in a separate thread allows the GUI to remain responsive so users can type messages, click buttons, and interact with the application while messages are received simultaneously.

3. Which components from Assignment 5 were reused?

The following components from Assignment 5 were reused:

- TCP socket communication
- Multi-threaded server implementation
- Client connection handling
- Broadcast messaging

-
- Private messaging
 - Online user management
 - Chat history logging
 - User join and leave notifications
 - Server-side message processing

Only the terminal-based client interface was replaced with a Tkinter GUI, while the core networking logic remained unchanged.

4. How did the GUI improve usability?

The graphical user interface made the application easier to use by replacing command-line interaction with buttons, text boxes, and an online users list. Messages are displayed in a scrollable chat window, users can send messages with a single click or by pressing Enter, connection status is clearly shown, and online users are displayed automatically. These improvements provide a more user-friendly and intuitive experience.

5. Suggest one future enhancement.

One possible future enhancement is to support **file sharing** between connected users. Users could send documents, images, or other files directly through the chat application. Additional enhancements could include message encryption, user authentication, emoji support, group chats, and message notifications to further improve security and usability.

9. Conclusion

This assignment successfully transformed the terminal-based TCP chat application developed in Assignment 5 into a graphical desktop application using Python Tkinter. The existing server implementation and networking logic were reused, while a user-friendly graphical interface was developed for client interaction. The application supports multiple clients, broadcast messaging, private messaging, online user management, join and leave notifications, and real-time communication using TCP sockets.